Paso 1 — Crear el proyecto y habilitar Broadcasting con Reverb.

```
composer create-project laravel/laravel emisor-receptor
cd emisor-receptor
```

Ejecutá el instalador de broadcasting con Reverb (esto agrega el paquete `laravel/reverb`, crea `config/broadcasting.php`, `routes/channels.php`, variables en `.env`, y el esqueleto de Echo):

```
php artisan install:broadcasting --reverb
```

El comando `install:broadcasting` con `--reverb` es la vía rápida y recomendada en Laravel 12 para dejar todo listo del lado servidor y cliente.

Recomendado para pruebas usar:

En el .env: QUEUE_CONNECTION=sync

Laravel emite los broadcasts mediante jobs encolados, por eso la doc recomienda tener un queue worker ejecutándose, salvo que uses sync para pruebas.

Paso 2 — Crear el Event que se va a emitir (canal público)

La idea: tener un evento RemoteStyleUpdate que lleve texto y un CSS inline (string). Este evento será "broadcastable" vía Reverb y lo escucharemos con Echo en el receptor.

Crea el evento:

```
php artisan make:event RemoteStyleUpdate
```

Edita app/Events/RemoteStyleUpdate.php para que implemente ShouldBroadcast y exponga el payload:

```php
<?php

namespace App\Events;

use Illuminate\Broadcasting\Channel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;

class RemoteStyleUpdate implements ShouldBroadcast
{
    public string $newText;
    public string $newStyle;

    public function __construct(string $newText, string $newStyle)
    {
        $this->newText  = $newText;
        $this->newStyle = $newStyle;
    }

    // Canal PÚBLICO (no requiere auth)
    public function broadcastOn(): array
    {
        return [new Channel('style-channel')];
    }

    // (Opcional) nombre del evento en el frontend
    public function broadcastAs(): string
    {
        return 'RemoteStyleUpdate';
    }
}
```

Notas rápidas (según docs oficiales):

Los eventos que “viajan” por WebSockets deben implementar ShouldBroadcast y definir broadcastOn() (y opcionalmente broadcastAs() y broadcastWith() si querés personalizar nombre/payload).

Para canales públicos basta con Channel('nombre'); routes/channels.php solo se usa para autorizar privados/presencia.

Con Laravel 12 + Reverb, el comando install:broadcasting ya deja la app preparada para emitir por Reverb; luego el reverb:start levanta el servidor WebSocket. Debemos asegurarnos que el .env tengas BROADCAST_CONNECTION=reverb (el instalador lo deja así).

Fuente: laravel.com

Paso 3 — Rutas, controlador y vistas:

Rutas: editamos routes/web.php

```
use App\Http\Controllers\RemoteStyleController;
use Illuminate\Support\Facades\Route;

Route::get('/', fn() => redirect('/emisor'));

Route::get('/emisor',  [RemoteStyleController::class,'emisor'])->name('emisor');
Route::post('/emitir', [RemoteStyleController::class,'emitir'])->name('emitir');

Route::get('/receptor', [RemoteStyleController::class, 'receptor'])->name('receptor');
```

Nota: La "arquitectura" de broadcasting es: el backend emite eventos a canales; el cliente se suscribe a esos canales con Echo.
Fuente: laravel.com

Creamos el controlador:

```
php artisan make:controller RemoteStyleController
```

Editamos el código de nuestro controlador:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Events\RemoteStyleUpdate;

class RemoteStyleController extends Controller
{
    public function emisor()
    {
        return view('emisor');
    }
```

```
    public function receptor()
    {
        return view('receptor');
    }

    public function emitir(Request $request)
    {
        $data = $request->validate([
            'newText'  => ['required','string'],
            'newStyle' => ['nullable','string'],
        ]);

        broadcast(new RemoteStyleUpdate(
            $data['newText'],
            $data['newStyle'] ?? ''
        ));

        return back()->with('ok', 'Mensaje emitido correctamente.');
    }
}
```

Creamos la vista emisor:
`resources/views/emisor.blade.php`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Emisor</title>
  @vite(['resources/css/app.css','resources/js/app.js'])
  <style>
    body { font-family: system-ui, sans-serif; padding: 24px; }
    .wrap { max-width: 760px; margin: 0 auto; }
    .box  { border: 1px solid #ddd; border-radius: 12px; padding: 16px; }
    label { font-weight: 600; }
    textarea, input, button { width: 100%; padding: 10px; margin-top: 6px; }
    .row { display: grid; gap: 12px; }
  </style>
</head>
<body>
<div class="wrap">
  <h1>Emisor</h1>
```

```
  @if (session('ok'))
    <div style="background:#e7f7ee;border:1px solid
#b7e2c8;padding:8px;border-radius:8px;margin-bottom:12px;">
      {{ session('ok') }}
    </div>
  @endif
  @if ($errors->any())
    <div style="background:#fde8e8;border:1px solid
#f5c2c2;padding:8px;border-radius:8px;margin-bottom:12px;">
      {{ $errors->first() }}
    </div>
  @endif

  <form class="box" method="POST" action="{{ route('emitir') }}">
    @csrf
    <div class="row">
      <div>
        <label>Texto a mostrar</label>
        <input type="text" name="newText" placeholder="Ej: Ahora atendiendo: A-037"
required>
      </div>

      <div>
        <label>CSS inline (opcional)</label>
        <textarea name="newStyle" rows="6" placeholder="Ej:
background:#111;color:#fff;padding:24px;border-radius:16px;font-size:28px;text-align:
center;"></textarea>
      </div>

      <button type="submit">Emitir</button>
    </div>
  </form>

  <p style="margin-top:16px;">
    Abrí <code>/receptor</code> en otra pestaña/ventana para ver los cambios en
tiempo real.
  </p>
</div>
</body>
</html>
```

Creamos la vista receptor:
`resources/views/receptor.blade.php`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Receptor</title>
  @vite(['resources/css/app.css','resources/js/app.js'])
  <style>
    body { font-family: system-ui, sans-serif; padding: 24px; }
    .wrap { max-width: 760px; margin: 0 auto; }
    #target-div { padding: 20px; border: 1px dashed #bbb; border-radius: 12px; }
  </style>
</head>
<body>
<div class="wrap">
  <h1>Receptor</h1>
  <div id="target-div">Esperando mensaje...</div>
</div>

<script>
  document.addEventListener('DOMContentLoaded', function () {
    const targetDiv = document.getElementById('target-div');

    // Suscripción al canal público y escucha del evento
    window.Echo.channel('style-channel')
      .listen('.RemoteStyleUpdate', (e) => {
        // El nombre con punto (.) escucha usando broadcastAs()
        targetDiv.innerText = e.newText ?? '';
        targetDiv.style.cssText = e.newStyle ?? '';
      });
  });
</script>
</body>
</html>
```

Echo + Reverb: el instalador ya deja el "scaffolding" de Echo y variables VITE_REVERB_*. Manualmente, se configura en resources/js/bootstrap.js con broadcaster: 'reverb' y las env VITE_REVERB_APP_KEY, VITE_REVERB_HOST, VITE_REVERB_PORT, etc. Reverb usa el protocolo Pusher en el cliente (pusher-js).

Posteriormente debemos recompilar los assets: Corriendo npm install npm run dev

Echo se instancia en el JS de tu app (por defecto en resources/js/bootstrap.js) y necesita compilarse con Vite para que el navegador lo use.

Ahora ya podriamos:

- *Abrir `/receptor` en una pestaña.*
- *Ir a `/emisor`, escribir texto y (opcional) CSS, y **Enviar**.*
- *Ver cómo el receptor **actualiza en vivo**.*

Compilar los assets

```
npm install
npm run dev
```

Iniciar los servicios

En **una terminal**:

```
php artisan serve --host=0.0.0.0 --port=8000
```

En **otra terminal**:

```
php artisan reverb:start
```

Ejemplo de archivo .env

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:QgLh/NpqcIZli3bVGGzWF3qoTOfILmFMlMekjynzZ4k=
APP_DEBUG=true
APP_URL=http://localhost

APP_LOCALE=en
APP_FALLBACK_LOCALE=en
```

```
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

# Base de datos (SQLite simple para dev)
DB_CONNECTION=sqlite


SESSION_DRIVER=file
SESSION_LIFETIME=120

FILESYSTEM_DISK=local
QUEUE_CONNECTION=sync
CACHE_STORE=file

# Mail en logs
MAIL_MAILER=log
MAIL_HOST=127.0.0.1
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_FROM_ADDRESS="hello@example.com"
MAIL_FROM_NAME="${APP_NAME}"

# Broadcasting con Reverb
BROADCAST_CONNECTION=reverb

REVERB_APP_ID=622505
REVERB_APP_KEY=uoj2o6k8snqvfgox3ewd
REVERB_APP_SECRET=sa98tgrijzlm7janzkhr
REVERB_HOST=localhost
REVERB_PORT=8080
REVERB_SCHEME=http

# Echo (Vite)
VITE_APP_NAME="${APP_NAME}"
```

```
VITE_REVERB_APP_KEY="${REVERB_APP_KEY}"
VITE_REVERB_HOST="${REVERB_HOST}"
VITE_REVERB_PORT="${REVERB_PORT}"
VITE_REVERB_SCHEME="${REVERB_SCHEME}"
```

Opcional pruebas con Sweet Alert:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Receptor</title>
  @vite(['resources/css/app.css','resources/js/app.js'])
  <style>
    body { font-family: system-ui, sans-serif; padding: 24px; }
    .wrap { max-width: 760px; margin: 0 auto; }
    #target-div { padding: 20px; border: 1px dashed #bbb; border-radius: 12px; }
  </style>
  <!-- SweetAlert2 por CDN -->
  <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
</head>
<body>
<div class="wrap">
  <h1>Receptor</h1>
  <div id="target-div">Esperando mensaje...</div>
</div>

<script>
  document.addEventListener('DOMContentLoaded', function () {
    const targetDiv = document.getElementById('target-div');

    // Suscripción al canal público y escucha del evento
    window.Echo.channel('style-channel')
      // Con broadcastAs('RemoteStyleUpdate') se escucha con un punto adelante
```

```
      .listen('.RemoteStyleUpdate', (e) => {

        // Actualizar el contenedor como ya hacías
        targetDiv.innerText = e.newText ?? '';
        targetDiv.style.cssText = e.newStyle ?? '';

        // Disparar un SweetAlert (toast no bloqueante)
        Swal.fire({
          toast: false,
          position: 'center',
          icon: 'warning',
          title: e?.newText ? `Nuevo mensaje: ${e.newText}` : 'Nuevo mensaje
recibido',
          showConfirmButton: false,
          timer: 3000,
          timerProgressBar: true
        });
      });
  });
</script>
</body>
</html>
```

ejemplo de css para probar pasar:

```
width: 80%;
 max-width: 600px;
 margin: 40px auto;
 padding: 20px;
 background-color: #e02424; /* Rojo fuerte */
 border: 2px solid #b91c1c; /* Borde rojo oscuro */
 border-radius: 12px;
 box-shadow: 0 4px 15px rgba(220, 38, 38, 0.5); /* Sombra roja */
 color: #fff; /* Texto blanco */
 font-family: "Segoe UI", sans-serif;
 text-align: center;
 transition: transform 0.2s ease, box-shadow 0.2s ease;
```